Deserialization

👉 Deserialization

- Deserialization is the process of reconstructing an object from its stored representation in a file.

Example

- The code shows how to deserialize a Student object that was previously saved to a file.

The Student Class

```java
class Student implements Serializable {
  private int id;
  private String name;
  private int age;

  // Constructor to initialize student properties
  public Student(int id, String name, int age) {
      this.age = age;
      this.id = id;
      this.name = name;
  }

  // Method to display student information
  public void disp(){
    System.out.println("id "+id);
    System.out.println("name "+name);
    System.out.println("age "+age);
  }
}
```

- The class implements Serializable interface, which marks it as eligible for serialization or deserialization.

👉 The Main Class (LaunchIO)

```java
public class LaunchIO {
    public static void main(String[] args){
        try {
            //Create FileInputStream to read from the file
            FileInputStream fis = new FileInputStream("D:\\telusko\\InputOutput\\serial.txt");

            //Create ObjectInputStream to read objects
            ObjectInputStream ois = new ObjectInputStream(fis);

            //Read and cast the object to Student type
            Student student = (Student) ois.readObject();

            //Display the deserialized object information
            student.disp();

            //Close the streams
            ois.close();
            fis.close();
        }
        // Exception handling
        catch (FileNotFoundException ex) {
          ex.printStackTrace();
        }
        catch (IOException e) {
          e.printStackTrace();
        }
        catch (ClassNotFoundException ex) {
          ex.printStackTrace();
        }
    }
}
```

Output:

```
id 1
name Rohan
age 16
```

👉 Explanation

- **Implementing Serializable**: The Student class implements the Serializable interface, which is required for any class whose objects you want to serialize or deserialize.

- **Reading the File**: A FileInputStream connects to the file where the serialized object is stored.

- **Creating Object Stream**: An ObjectInputStream wraps the file stream to handle reading objects.

- **Deserializing the Object**: The readObject() method reads the serialized object from the file. Since this method returns an Object type, we need to cast it to Student.

- **Using the Deserialized Object**: After deserialization, the student object contains all the values it had when it was serialized (id=1, name="Rohan", age=16).

- **Exception Handling**: The code handles three possible exceptions:
  - FileNotFoundException: If the file doesn't exist
  - IOException: For general I/O errors
  - ClassNotFoundException: If the class of the serialized object cannot be found